

1. EXECUTIVE SUMMARY

This report documents the design, algorithmic flow, and execution testing of a CLI-based **Number Guessing Game** developed in Python 3. The interactive application incorporates dynamic difficulty scaling, state validation, custom score computation, and proximity hint heuristics to optimize user feedback during runtime.

2. OBJECTIVES & SCOPE

- Randomization:** Utilize Python's standard pseudo-random number generator to seed secrets dynamically per session.

Difficulty Mode	Range [Min, Max]	Max Attempts	Bonus Points
Easy	1 to 50	10	+5
Medium	1 to 100	7	+10
Hard	1 to 200	5	+20

Input Validation & Exception Safety: Wrap CLI input parsing in structured try-except handlers to prevent process crashing on non-numeric inputs.

3. SYSTEM REQUIREMENTS

Component	Minimum Requirement	Recommended Specification
Operating System	Windows 10 / Linux / macOS	Cross-platform compatible
Runtime Environment	Python 3.6+	Python 3.10+
Memory & Storage	512 MB RAM / 10 MB Disk	1 GB RAM / 50 MB Disk
User Interface	Command Line Terminal	VS Code / PyCharm / Windows Terminal

4. SOURCE CODE IMPLEMENTATION

```
import random

print("=====")
print("    NUMBER GUESSING GAME")
print("=====")

print()
print("Choose Difficulty:")
print("1. Easy")
print("2. Medium")
print("3. Hard")

choice = input("Enter your choice (1/2/3): ")

if choice == "1":
    minimum = 1
    maximum = 50
    attempts = 10
    difficulty = "Easy"

elif choice == "2":
    minimum = 1
    maximum = 100
    attempts = 7
    difficulty = "Medium"

elif choice == "3":
    minimum = 1
    maximum = 200
    attempts = 5
    difficulty = "Hard"

else:
    print("Invalid choice.")
    exit()

number = random.randint(minimum, maximum)

print()
print("Difficulty:", difficulty)
print("Guess a number between", minimum, "and", maximum)
print("You have", attempts, "attempts.")
print()

score = 0

while attempts > 0:

    try:
        guess = int(input("Enter your guess: "))
    except ValueError:
        print("Please enter a valid number.")
        continue

    if guess < minimum or guess > maximum:
        print("Please enter a number within the given range.")
        continue

    if guess == number:
        score = attempts * 10

        if difficulty == "Hard":
            score += 20
        elif difficulty == "Medium":
            score += 10
        else:
            score += 5
```

```

print()
print("Congratulations!")
print("You guessed the correct number.")
print("The correct number was:", number)
print("Your score:", score)
break

elif guess < number:
    print("Too low! Try a higher number.")

else:
    print("Too high! Try a lower number.")

difference = abs(guess - number)

if difference <= 5:
    print("Hint: You are very close!")
elif difference <= 15:
    print("Hint: You are close!")
else:
    print("Hint: You are far away.")

attempts -= 1
print("Attempts remaining:", attempts)
print()

else:
    print()
    print("Game Over!")
    print("The correct number was:", number)
    print("Your score: 0")

print()
print("=====")
print("      GAME FINISHED")
print("=====")

```

5. TERMINAL EXECUTION TRACE & OUTPUT

```
23:20 23:20 82%
← TAB ⋮
=====
NUMBER GUESSING GAME
=====

Choose Difficulty:
1. Easy
2. Medium
3. Hard
Enter your choice (1/2/3): 2

Difficulty: Medium
Guess a number between 1 and 100
You have 7 attempts.

Enter your guess: 50
Too low! Try a higher number.
Hint: You are far away.
Attempts remaining: 6

Enter your guess: 55
Too low! Try a higher number.
Hint: You are far away.
Attempts remaining: 5

Enter your guess: 66
Too low! Try a higher number.
Hint: You are close!
Attempts remaining: 4

Enter your guess: 88
Too high! Try a lower number.
Hint: You are close!
Attempts remaining: 3

Enter your guess: 66
Too low! Try a higher number.
Hint: You are close!
Attempts remaining: 2

Enter your guess: 77
Too high! Try a lower number.
Hint: You are very close!
Attempts remaining: 1

Enter your guess: 68
Too low! Try a higher number.
Hint: You are close!
Attempts remaining: 0

Game Over!
The correct number was: 76
Your score: 0

=====
GAME FINISHED
=====

[Program finished]
```

6. CONCLUSION & SCOPE FOR IMPROVEMENT

The Python application meets all core requirements of a structured CLI software assignment. It demonstrates clean syntax, algorithmic branching, user input sanitization, and variable state monitoring.

Future Enhancement Roadmap:

- **High Score Persistence:** Save high scores locally using CSV or SQLite database bindings.
- **Graphical Interface:** Migrate command line interface into a desktop Tkinter or PyQt GUI.

Multiplayer Mode: Allow two players to compete in alternating turned-based rounds.

8. STUDENT DETAILS

Name: D.Vishnupriya

Roll No:26bai10614

Faculty name: S. Hariharasitaraman

Slot: B21 +B22+B23+E14+E21+E22